

Login simples com PocketBase e Alpine.js

Duas páginas, um formulário e uma autenticação. Vamos construir cada ideia em uma etapa.

O que vamos construir

O navegador mostra as páginas. Alpine.js lê os campos e responde aos eventos. PocketBase confere a conta do aluno no servidor.

As cinco etapas

ETAPA	CONCEITO PRINCIPAL	BRANCH
01	HTML e navegação entre páginas	etapa-01-html-estatico
02	Formulário reativo, sem autenticação	etapa-02-alpine
03	Login real com PocketBase	etapa-03-login-pocketbase
04	Confirmar sessão e sair	etapa-04-sessao-e-saida
05	Organizar os arquivos	etapa-05-arquivos-organizados

Como usar este guia

Abra a branch de cada etapa, leia os comentários nos arquivos e faça os experimentos. Cada branch mostra o código completo daquele momento da aula.

Público: alunos iniciantes em HTML, CSS e JavaScript.

Antes de começar

O aluno informa e-mail e senha. A página envia esses valores ao PocketBase. Se o servidor aceitar a conta, o navegador abre a área do aluno.

```
aluno -> login.html -> Alpine.js -> PocketBase -> logado.html
```

Papel de cada tecnologia

TECNOLOGIA	PAPEL
HTML	Cria as duas páginas, os campos e os botões.
CSS	Define aparência e espaçamento.
Alpine.js	Liga valores dos campos ao JavaScript e responde aos eventos.
PocketBase	Guarda contas e valida as credenciais no servidor.

Prepare o PocketBase

- Baixe o PocketBase para seu sistema e extraia o executável.
- Inicie o servidor com o comando abaixo.
- Abra <http://127.0.0.1:8090/> e crie uma conta de administrador.
- Crie uma coleção do tipo Auth chamada users e cadastre um aluno com e-mail e senha.

```
.\pocketbase.exe serve
```

Abra as páginas

Mantenha o PocketBase rodando. Em outro terminal, na pasta do projeto:

```
py -m http.server 5500  
http://127.0.0.1:5500/login.html
```

Use o mesmo endereço

Neste projeto, use 127.0.0.1 no navegador e em js/pocketbase.js. localhost e 127.0.0.1 podem ter armazenamentos de login separados.

Etapa 1 | HTML estático

Branch: etapa-01-html-estatico

As duas páginas já existem. O objetivo é reconhecer os elementos HTML e entender que um link apenas navega.

O que observar

- h1 identifica o título principal.
- label descreve um campo de input.
- input recebe o texto digitado.
- a com href aponta para outra página.

```
<label for="email">E-mail</label>
<input id="email" type="email">
<a href="logado.html">Ver a segunda página</a>
```

O link não faz login

Os campos são visuais nesta etapa. A página logado.html pode ser aberta diretamente, mesmo sem e-mail ou senha.

Faça você mesmo

- Clique em Ver a segunda página sem digitar nada.
- Abra logado.html diretamente na barra de endereços.
- Altere o título h1 e observe a página no navegador.

Pense antes de avançar

O que teria de acontecer para o programa conferir uma conta de aluno? Primeiro vamos aprender a reação do formulário; depois enviaremos dados ao servidor.

Etapa 2 | Formulário com Alpine.js

Branch: etapa-02-alpine

Alpine.js liga os campos a variáveis. O envio chama uma função, que mostra uma mensagem na própria página. Ainda não existe autenticação.

Quatro atributos importantes

- x-data define os dados e as funções desta parte do HTML.
- x-model acompanha o valor digitado.
- @submit.prevent chama enviar() sem recarregar a página.
- x-show e x-text mostram a mensagem apropriada.

```
<main x-data="formulario()">
  <form @submit.prevent="enviar">
    <input x-model="email">
    <p x-show="mensagem" x-text="mensagem"></p>
  </form>
</main>
```

O que o código faz

Ele lê os campos e exibe uma confirmação local. Não compara senhas, não envia dados ao servidor e não abre a área do aluno. A senha é limpa e não aparece na mensagem.

Faça você mesmo

- Envie os campos vazios e observe o erro.
- Use um e-mail com formato válido e uma senha qualquer.
- Veja a mensagem e confirme que a senha não aparece nela.

Conceito-chave

O estado de x-data vive na página. Ele serve para controlar a interface; uma mensagem de sucesso na tela não comprova a identidade de ninguém.

Etapa 3 | Login com PocketBase

Branch: etapa-03-login-pocketbase

Agora o formulário chama o PocketBase. A coleção users deve ser do tipo Auth e conter o aluno que você vai usar no experimento.

O endereço do servidor

```
window.pb = new PocketBase('http://127.0.0.1:8090');
```

O SDK é uma biblioteca que prepara as chamadas à API e mantém o estado do login no navegador.

A chamada de autenticação

```
await window.pb
  .collection('users')
  .authWithPassword(this.email, this.senha);
```

- collection('users') escolhe a coleção das contas dos alunos.
- authWithPassword pede ao servidor que confira e-mail e senha.
- await aguarda a resposta antes do redirecionamento.
- try/catch separa o resultado aceito do erro.

Após o login, o SDK guarda um token e um registro de usuário. A página abre logado.html. Esta etapa ainda permite abrir logado.html diretamente sem uma confirmação adicional.

Token em poucas palavras

O servidor emite um token após aceitar as credenciais. O SDK o guarda no navegador e o envia em pedidos autenticados posteriores.

Faça você mesmo

- Tente entrar com uma senha errada.
- Entre com o aluno cadastrado no painel do PocketBase.
- Abra logado.html em uma janela anônima e observe o limite desta etapa.

Etapa 4 | Confirmar a sessão e sair

Branch: etapa-04-sessao-e-saida

Ao abrir a área do aluno, usamos dois passos: primeiro verificamos o token salvo localmente; depois pedimos ao PocketBase para confirmá-lo.

```
if (!window.pb.authStore.isValid) {  
  window.location.replace('login.html');  
  return;  
}  
await window.pb.collection('users').authRefresh();
```

- isValid verifica apenas se existe um token local ainda não expirado.
- authRefresh faz uma chamada ao servidor e atualiza o token e o registro.
- Enquanto a resposta não chega, a tela diz Verificando login.
- Se a confirmação falhar, o código limpa o estado e volta ao login.

Para sair, limpamos o estado salvo pelo SDK:

```
window.pb.authStore.clear();  
window.location.replace('login.html');
```

Onde fica a proteção dos dados?

O navegador pode baixar os arquivos HTML e JavaScript. Para dados privados de outras coleções, configure as regras de API no PocketBase. Uma regra de leitura só para usuários autenticados é `@request.auth.id != ""`.

Faça você mesmo

- Entre e atualize a página: o servidor confirma a sessão novamente?
- Clique em Sair e abra logado.html diretamente.
- Abra uma janela anônima e tente visitar logado.html sem login.

Etapa 5 | Organizar os arquivos

Branch: etapa-05-arquivos-organizados e versão final: main

Movemos o CSS e o JavaScript para arquivos próprios. As funções de login e da área do aluno são as mesmas da etapa 4; só mudou onde elas estão escritas.

```
login.html
logado.html
css/estilo.css
js/pocketbase.js
js/login.js
js/logado.js
```

Responsabilidade de cada arquivo

- login.html contém o formulário; logado.html contém a área do aluno.
- css/estilo.css guarda estilos usados nas duas páginas.
- js/pocketbase.js aponta para o servidor.
- js/login.js envia as credenciais.
- js/logado.js confirma a sessão, mostra o e-mail e permite sair.

Por que a ordem dos scripts importa?

```
<script defer src="../../../pocketbase.umd.js"></script>
<script defer src="js/pocketbase.js"></script>
<script defer src="js/login.js"></script>
<script defer src="../../../alpine.min.js"></script>
```

O navegador lê o HTML e depois executa os scripts defer na ordem escrita. O SDK precisa existir antes de js/pocketbase.js; a função da página precisa existir antes de Alpine iniciar.

Volte a uma etapa

Use git switch etapa-01-html-estatico para ver o primeiro exemplo. Avance uma branch por vez e use git switch main para voltar à versão final.

Verificar e resolver problemas

Se o login não funcionar, siga o caminho da requisição e confira uma parte por vez.

- O painel abre em `http://127.0.0.1:8090/_/?`
- A coleção se chama exatamente `users` e é do tipo `Auth`?
- O aluno foi cadastrado em `users`, com e-mail e senha?
- A página abriu por `http://127.0.0.1:5500/login.html?`
- `js/pocketbase.js` usa o endereço certo?
- Há internet para carregar `Alpine.js` e o `SDK` pelos links `CDN`?

Verificação manual do fluxo

- Senha errada: permanece no login e mostra uma mensagem.
- Credenciais certas: abre a área do aluno e mostra o e-mail.
- Atualização da área: exibe Verificando login e consulta o servidor.
- Sair: limpa o login e volta ao formulário.
- Acesso direto sem sessão: retorna ao login.

Sintomas frequentes

SINTOMA	CAUSA PROVÁVEL	CONFIRA
Não conecta	PocketBase parado ou endereço diferente	Terminal e <code>js/pocketbase.js</code>
Login falha	Dados ou coleção incorretos	Registro em <code>users</code>
Estilo não aparece	Caminho do CSS incorreto	<code>css/estilo.css</code> no HTML
Tela invisível	Alpine ou outro script não carregou	Internet e console

Uma mudança por vez

Leia a primeira mensagem vermelha no console do navegador, compare os caminhos dos arquivos e corrija um problema por vez.

Glossário e desafios

Palavras usadas no projeto

- API: interface pela qual um programa pede dados ou serviços a outro.
- Autenticação: conferir quem está tentando entrar.
- Coleção Auth: coleção especial do PocketBase para contas de usuário.
- Estado: valores em uso na tela, como e-mail, erro ou mensagem.
- Token: credencial emitida pelo servidor após o login.
- Redirecionamento: abrir outra URL por link ou JavaScript.
- Regra de API: condição aplicada pelo servidor ao acesso a uma coleção.

Desafios para continuar

- Adicione um nome ao registro do aluno e mostre-o na área logada.
- Mude o texto do botão durante a espera pela autenticação.
- Altere a cor principal em `css/estilo.css` e observe as duas páginas.
- Crie uma coleção de exemplo e permita leitura apenas a usuários autenticados.

Documentação oficial

[Autenticação no PocketBase](#)

[Regras de API do PocketBase](#)

[x-data no Alpine.js](#)

Ao terminar

Você terá visto a diferença entre uma página estática, um formulário reativo, uma autenticação no servidor e uma regra que protege dados.